Code Together, Learn Faster: The XX Guide to Social Coding

How we’re evolving our development process to maximize learning, speed, and transparency



NotebookLM

To build the future of XX, we we must evolve how we build.

We are changing almost every aspect of how we build the XX platform. To succeed, we must optimize our processes around three key objectives.



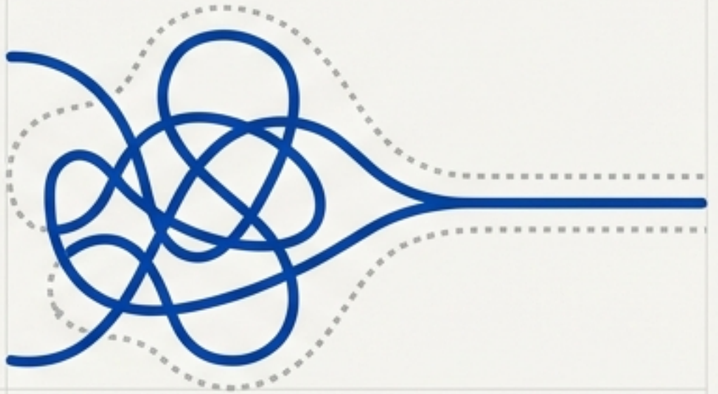
Maximize for Learning

Learn by doing in live, shared, and transparent codebases. Use transparency to help each other and accelerate progress.



Increase Speed

Move fast, learn, and adjust. Plan to fail, but fail early (and learn from it).



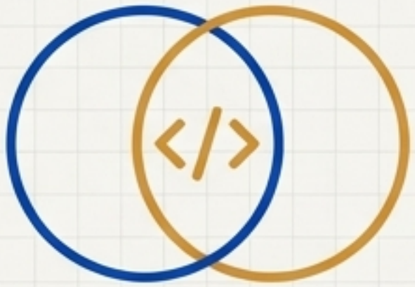
Eliminate Waste

Collaborate through social coding to reduce administrative overhead and duplicated efforts.

NotebookLM

Our solution: Embracing Social Coding.

Social coding is a development philosophy that uses transparency and collaboration to improve our work and our environment. It directly benefits us in three ways:



1. Amplify Knowledge Exchange

Discover and reuse code or solutions that you might otherwise duplicate. Share and implement new ideas with your colleagues.



2. Elevate Code Quality

When all developers can see and comment on code changes, there is an added incentive to get it right. A larger audience increases the likelihood that someone knows the answer to a problem.



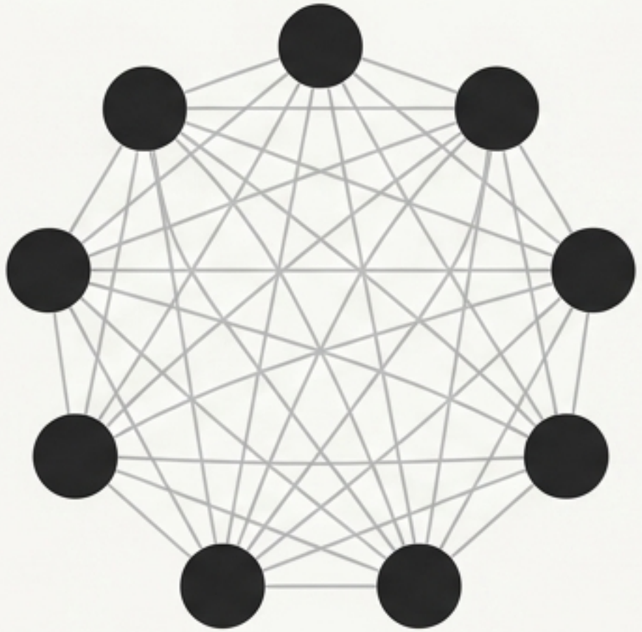
3. Foster a Stronger Culture

It encourages a fun work environment, promotes peer recognition, and builds a meritocracy—all essential to our DevOps culture.

NotebookLM

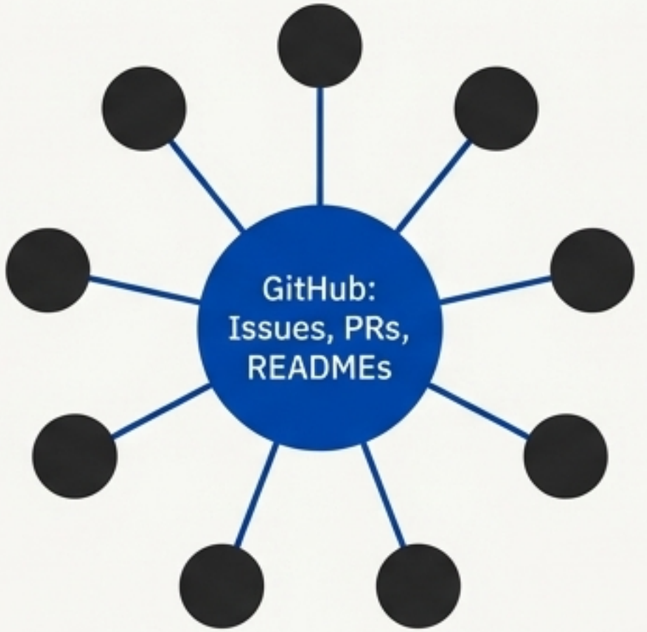
We’re moving from a complete graph to a small-world network.

The Old Way: Complete Graph



High communication overhead. Growth is unsustainable. Information is siloed in direct conversations.

Social Coding: Small-World Network



Linear or sublinear overhead. Information is public and searchable. Teams self-organize.

Key Takeaway: By making our work transparent, we reduce the need for direct person-to-person communication and untangle cross-team dependencies.

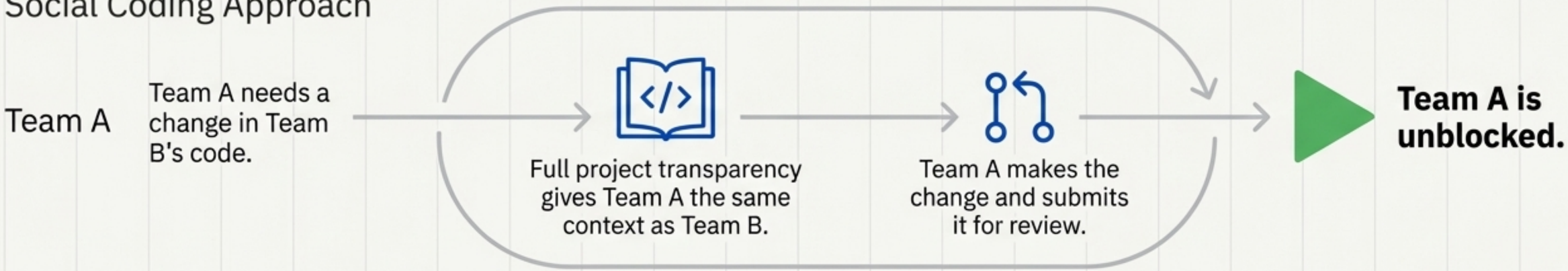
NotebookLM

Empowering teams to contribute, not just request.

Traditional Approach



Social Coding Approach

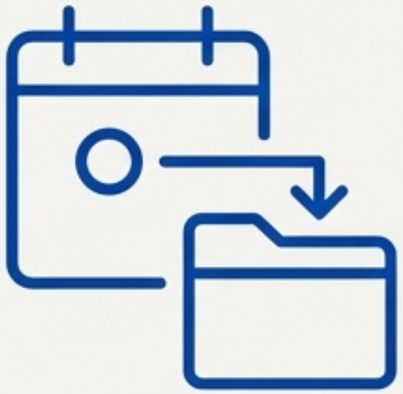


"Social coding empowers Team A to make and submit the change rather than wait for Team B to prioritize the work."

NotebookLM

The daily commit is our heartbeat.

To ensure efficient and transparent collaboration, we are adopting two new expectations for our daily work rhythm.



Expectation 1: Commit Daily to `master/trunk`

It is OK to commit unfinished code. We expect all developers to commit to the codebase daily when they are working on it.

Why? This provides continuous visibility into progress and prevents large, complex integration challenges.

Further Reading: Why I love trunk-based development?



Expectation 2: Review Diffs for 5-10 Minutes Daily

We expect all developers to spend 5-10 minutes every day browsing their own and others' commit-diffs on GitHub.

Why? To engage when you observe misunderstandings, design degradation, or potential bugs. This is how we practice collective code ownership.

NotebookLM

The README is your project's front door.

The use case: A developer is woken at 03:00 to fix a critical production issue on a codebase they've never seen. A great README gives them the context to understand the situation, fix the problem, and release a patch as fast as possible.



Running Environments: Direct links to the service running in common environments.



Documentation: Links to API docs (e.g., Swagger) and wikis (Confluence/GitHub).



CI/CD Pipeline: Link to the build and deploy pipeline.



Status Badges: Embedded, live badges for build status, version tag, and SNYK vulnerability status.



Live Service Info: Links to `/health` and `/info` endpoints displaying version and dataset info.



Admin Links: Links to the SCM repo and project boards (GitHub & Jira).



Developer Onboarding: Instructions for how to build and test the project locally.

NotebookLM

Our Social Coding Toolkit.



The Source of Truth

Our new source code repository for hero codebases. We use its features—Issues, PRs, and READMEs—to plan, track, and communicate around the code.



The Real-Time Hub

For real-time collaboration that is archived and searchable. Group channels allow new contributors to ask questions to the whole team, not just individuals.

MURAL

The Creative Space

A shared whiteboard for brainstorming, retrospectives, and virtual sticky notes where ideas can be generated quickly and reorganized later.

NotebookLM

Managing our work in GitHub.

What

Jira

Cross-repository, time-based functional view.

How

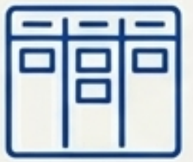
Technical, codebase-specific daily tasks.



Labels: Used for non-functional requirements (e.g., SLAs, multi-tenancy), functional milestones, and issue types (e.g., bug, `new-feature`, `critical_bug`).



Milestones: Used to organize the backlog by urgency and timeframe (e.g., `Blue sky`, `Next tasks`, `V1.2-release`).



Projects: Used for an up-to-date view of the current sprint, including an explicit Definition-of-Done and verification status.

NotebookLM

Our rules for technical governance.



Managing Dependencies

- **Scanning**: All codebases will use SNYK and GitHub dependency analysis to expose known security vulnerabilities.
- **Vulnerability Policy**: Patchable vulnerable dependencies must be updated immediately and are not allowed to remain un-patched for more than 2 patch-releases.
- **Freshness Policy**: In general, dependencies should not be more than 2 feature-releases behind the latest version without special dispensation.



Releasing Features Safely

- **Feature Toggles**: All new business features on production codebases will be wrapped in Unleash feature toggles so they never block a release.
- **Database Migrations**: Features requiring database changes will use Flyway to auto-migrate existing datasources.
- **Cross-Service Features**: For features spanning multiple services, we will use Branch by Abstraction as a ‘safety-net’ to implement old and new logic side-by-side.

NotebookLM

Shipping with clarity and confidence.

Signaling Changes with Semantic Versioning



- All modules will actively use semantic versioning to signal the state of our releases.
- We will release very often. Typically, a new feature release is created for each issue that has been completed and validated.
- More frequent patch-releases can be used to enable transparency and collaboration with product teams.

Our Definition of Done

In XX, ‘Done’ means **verified in the DEVTEST environment.**

This ensures we validate our code in a real microservice environment, catching issues that can only be seen post-deployment.

This ensures we validate our code in a real microservice environment, catching issues that can only be seen post-deployment. It forces us to learn the real-world, production aspects of our changes, making us better engineers.

NotebookLM

Our weekly rhythm keeps us aligned.

M



Monday Commitments

Kickstarts the week. A 15-minute discussion to review our goals and decide what to focus on. The list is posted on Slack for transparency.

F



Friday Wins

We share and celebrate this week’s achievements. Wins are shared on our team channel and broadcast to the wider organization on Slack.

NotebookLM

The Way Forward.

This is a mindset, not just a process. By prioritizing **Learning**, **Speed**, and **Transparency**, we empower ourselves to build better software, faster.

Experiment. Collaborate. Own the process.
Let’s build better, together.

For questions, discussions, and process suggestions, join the conversation in the `#social-coding-dev` Slack channel.

NotebookLM